JSON Authentication Endpoint Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ JSON Authentication Endpoint Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use controlled accounts and approved request limits. Do not brute-force credentials or target other users.*

■ Executive Summary

JSON authentication endpoints must validate input types and structure consistently, reject malformed or ambiguous data, and apply the same authentication controls as other login paths. The supplied 17-page *json.pdf* catalogs variations involving nulls, data types, malformed JSON, duplicate keys, encoding, nested objects, and injection-like strings.

■ Core Testing Methodology

1. Confirm the endpoint and test scope.
2. Capture a valid baseline request with a controlled account.
3. Check missing, empty, and null required fields.
4. Check strings, numbers, booleans, arrays, and objects against the documented schema.
5. Check malformed JSON and invalid escapes.
6. Verify duplicate keys are rejected or handled consistently.
7. Check length, Unicode, and nesting limits.
8. Ensure errors do not reveal credentials or stack traces.
9. Use harmless fixtures or code review to validate injection defenses.
10. Verify rate limits within the approved budget and document results.

■ High-Value Bug Bounty Tips & Edge Cases

- Empty/null values must not be treated as valid credentials accidentally.
- Type coercion can cause inconsistent validation across layers.
- Reject unsupported nested structures and ambiguous duplicate keys.
- Normalize Unicode consistently and validate canonical values.
- Limit long inputs and nested structures.
- Unexpected fields must not allow clients to set roles or privileges.
- The source is a dense catalogue; this guide consolidates visible categories rather than claiming every example was exhaustively transcribed.

■■ Educational Checklists & Safe Examples

```
POST /api/auth/login HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "username": "[CONTROLLED-TEST-USERNAME]",
  "password": "[TEST-PASSWORD]"
}
```

- [ ] Required fields and data types are enforced.
- [ ] Duplicate keys and malformed JSON are handled safely.

- [ ] Input lengths and nesting are bounded.
- [ ] Errors do not reveal credentials or internals.
- [ ] Rate limits apply consistently.
- [ ] Privilege fields cannot be set through login input.

■■ Mitigation & Remediation

Validate requests against an explicit schema, reject malformed or ambiguous fields, bound lengths and nesting, use parameterized database operations, return generic authentication errors, avoid logging secrets, enforce rate limits, and keep roles server-controlled.

■ Source Notes

Based on the supplied 17-page *json.pdf*, titled as JSON tests for authentication endpoints. Visible pages list empty/null, numeric, boolean, array/object, special-character, malformed, missing/extra/repeated-key, Unicode, escape, whitespace, long-value, and control-character categories.

Educational use only. Respect scope and request limits.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.